



Data Structures & Algorithms: The Python Foundation

From Time Complexity to Linked Lists with Chai aur Code

Foundations of Efficiency

Time Complexity Analysis

Arrays & Linked Lists

Language is just a tool. Logic is the key.

The Philosophy: Ingredients, Containers, and Recipes



DATA

The Ingredients. Raw information like user inputs, integers, and strings. Just as you need milk and sugar to make tea, software requires raw data.



STRUCTURE

The Container. Context dictates storage. You wouldn't drink tea from a bucket. We choose Arrays (contiguous) or Linked Lists (scattered) based on the need for access versus modification.



ALGORITHM

The Recipe. The step-by-step instructions to process the data. Efficiency is determined by how well the recipe scales.

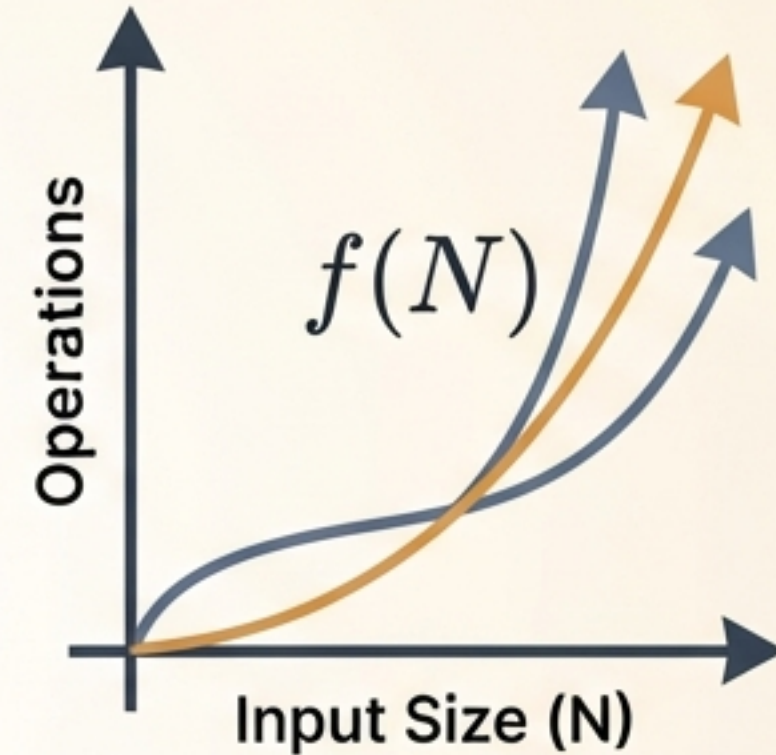
Measuring “Better”: Why We Don’t Use a Clock



Hardware varies. A supercomputer runs inefficient code faster than an old laptop runs efficient code.

Background processes skew results.

We cannot measure efficiency in seconds.



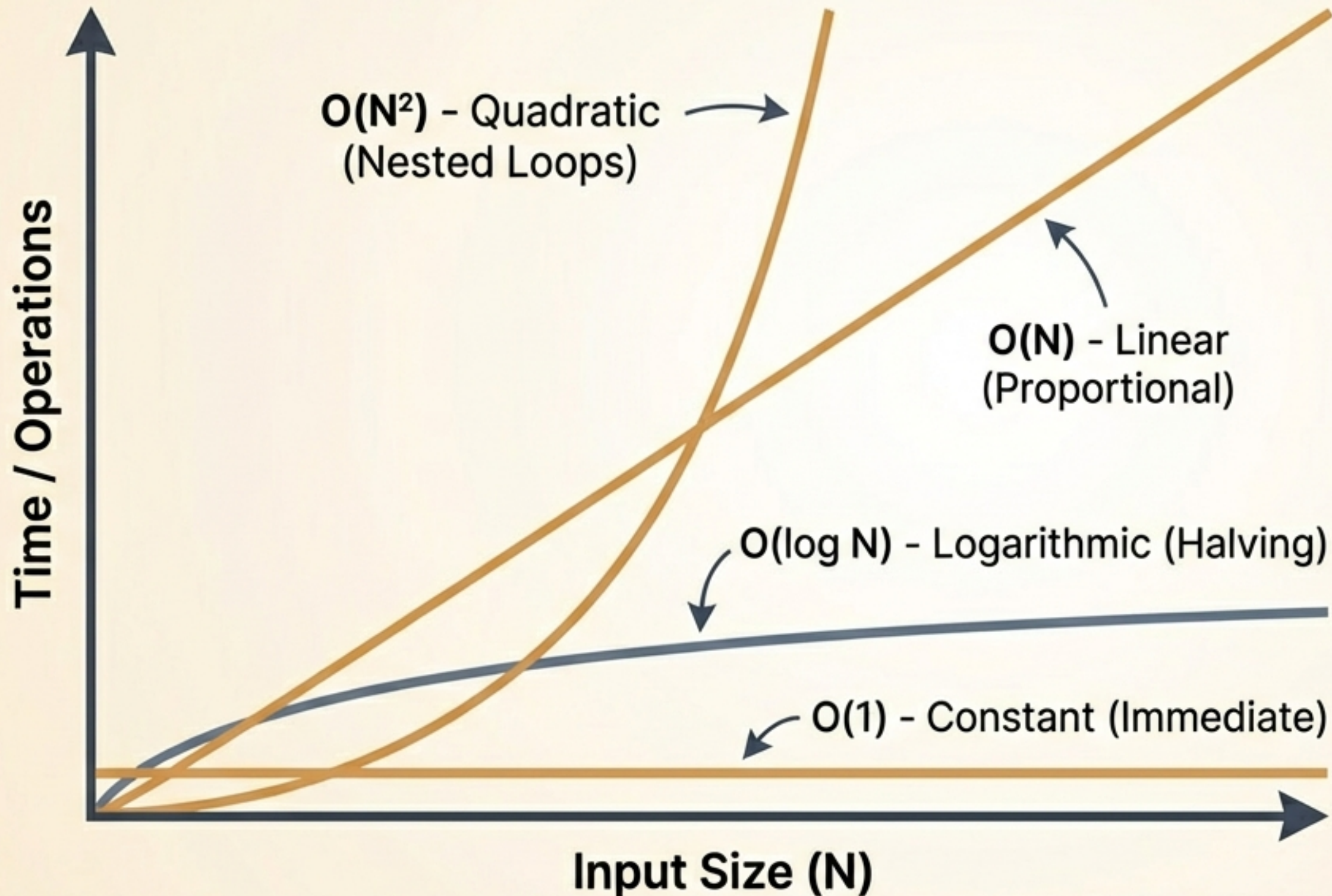
We count Operations relative to Input Size (N).

The Dominating Term Rule:

When N is large (e.g., 1,000,000), constants disappear.

- $2N + 5 \approx N$ (Linear)
- $N^2 + 100N \approx N^2$ (Quadratic)
- Focus on the Worst Case (Big O).

The Hierarchy of Complexity (Big O)



$O(1)$: Accessing an array index.

$O(\log N)$: Binary Search (Highly Efficient).

$O(N)$: Reading a book page by page.

$O(N^2)$: Comparing every item to every other item.

Takeaway: Always aim to move down the curve.

The 30-Second Rule: Judging Code at a Glance

```
print(x)  
y = 5 + x
```

No Loops = $O(1)$

Simple arithmetic, assignments, or comparisons take constant time.

```
for i in range(n):  
    print(i)
```

One Loop = $O(N)$

A single loop running 0 to N. Time grows proportionally to input.

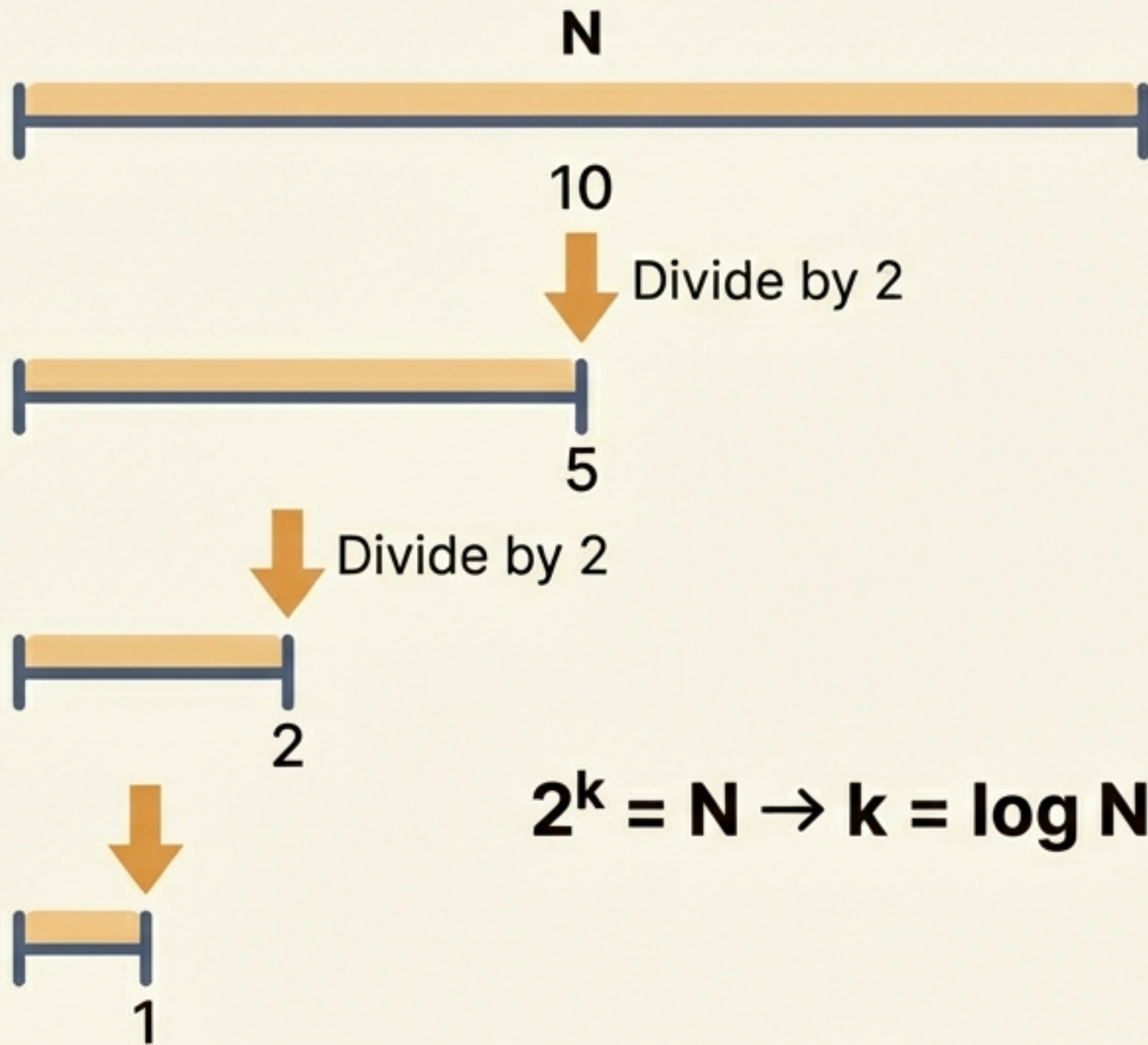
```
for i in range(n):  
    for j in range(n):  
        print(i, j)
```

Nested Loops = $O(N^2)$

A loop inside a loop. Runs $N \times N$ times. Avoid for large datasets.

Independent loops are added ($N + N$), not multiplied.

Understanding Logarithmic Time $O(\log N)$



```
while n > 1:  
    n = n / 2  
    # Input is halved every step
```

The Halving Logic: If a loop divides the input by a constant factor (like 2) every iteration, it is **Logarithmic**.

Insight: **Drastically faster** than $O(N)$. For massive data, it is nearly instant.

Arrays in Python: The Contiguous Container

The Array



- **Definition:** A collection of elements stored in contiguous memory locations.
- **Access:** $O(1)$. The computer calculates the exact address instantly.

The Python Difference

C/Java Arrays: Fixed size, homogeneous types.

Python Lists (`[]`):
Dynamic, heterogeneous (can mix int, string, float).

Python "array" Module:
`"import array"`. Strictly typed for efficiency.

The 'array' Module: Strict Efficiency

```
from array import array
# 'i' typecode enforces Integers only
vals = array('i', [1, 2, 3, 4, 5])

print(vals.buffer_info())
vals.append(6)
vals.remove(2)
vals.reverse()
```

Strict Type Enforcement
(e.g., only integers)

Compact, Homogeneous Storage

Low-Level Access:
(address, elements)

Performance Impact: # In-place reversal
Shifting elements takes time

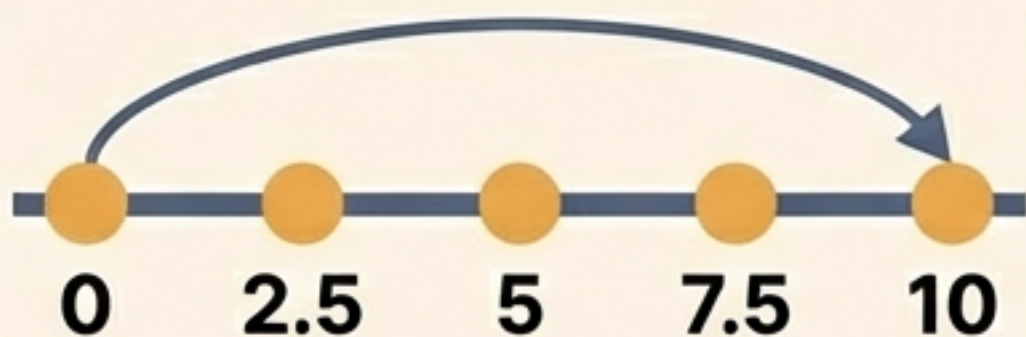


- **Creation:** Requires a typecode (e.g., 'i' for signed int, 'f' for float).
- **Iteration:** Loop via index 'range(len(vals))' or directly 'for x in vals'.
- **Constraint:** Unlike Lists, you cannot mix data types here.

NumPy: The Data Science Standard

Used when standard arrays aren't enough. Supports matrices and broadcasting.

linspace(start, stop, parts)



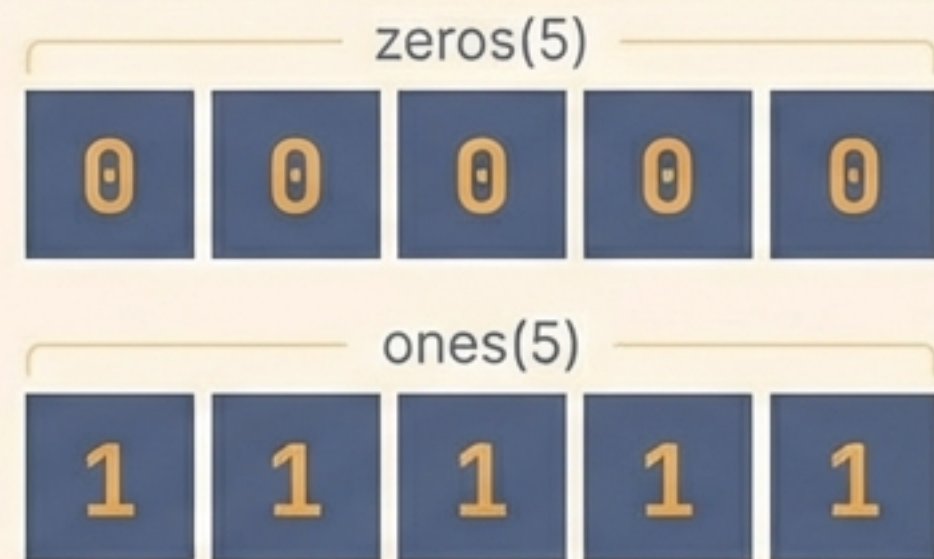
5 Equal Parts
(Start & Stop included)

arange(start, stop, step)



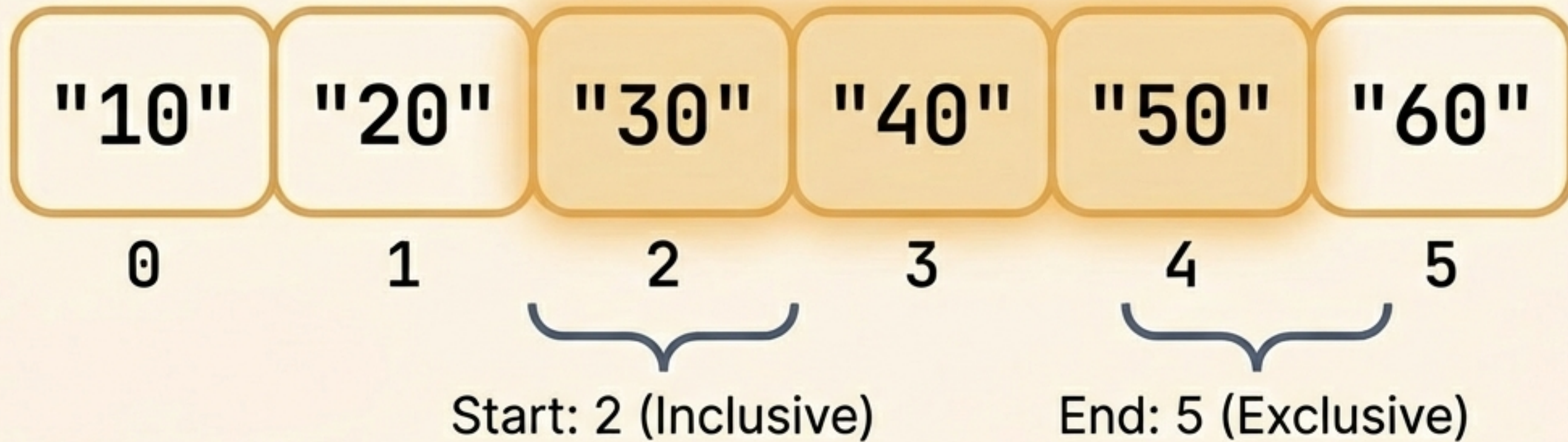
Steps of 2
(Stop point is excluded)

zeros(n) / ones(n)



Rapid Initialization

The Power of Slicing



Syntax: `arr[ Start : End : Step ]`

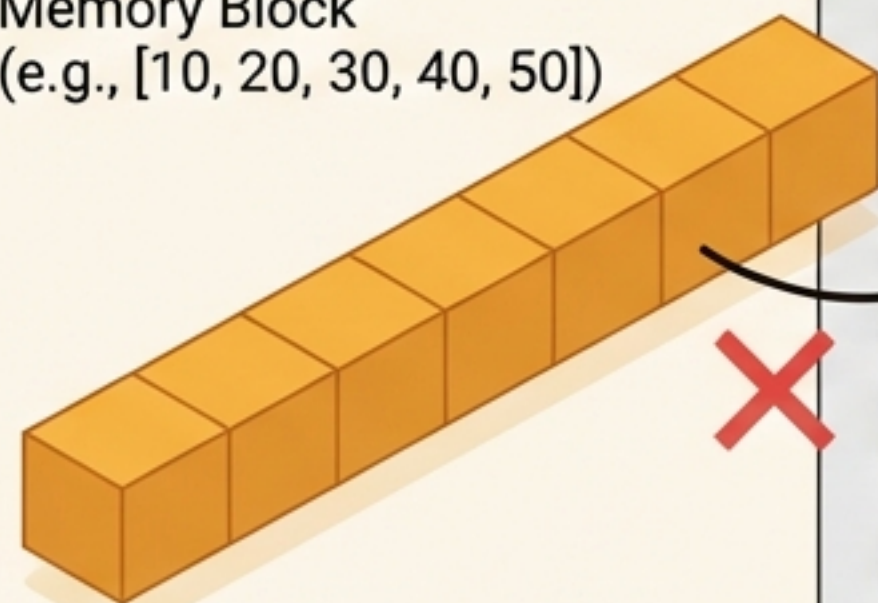
`arr[2:5]` → Returns `[30, 40, 50]`

`arr[::-1]` → Returns `[60, 50, 40, 30, 20, 10]` (Instant Reversal)

Insight: Slicing creates a new view or copy of the data.

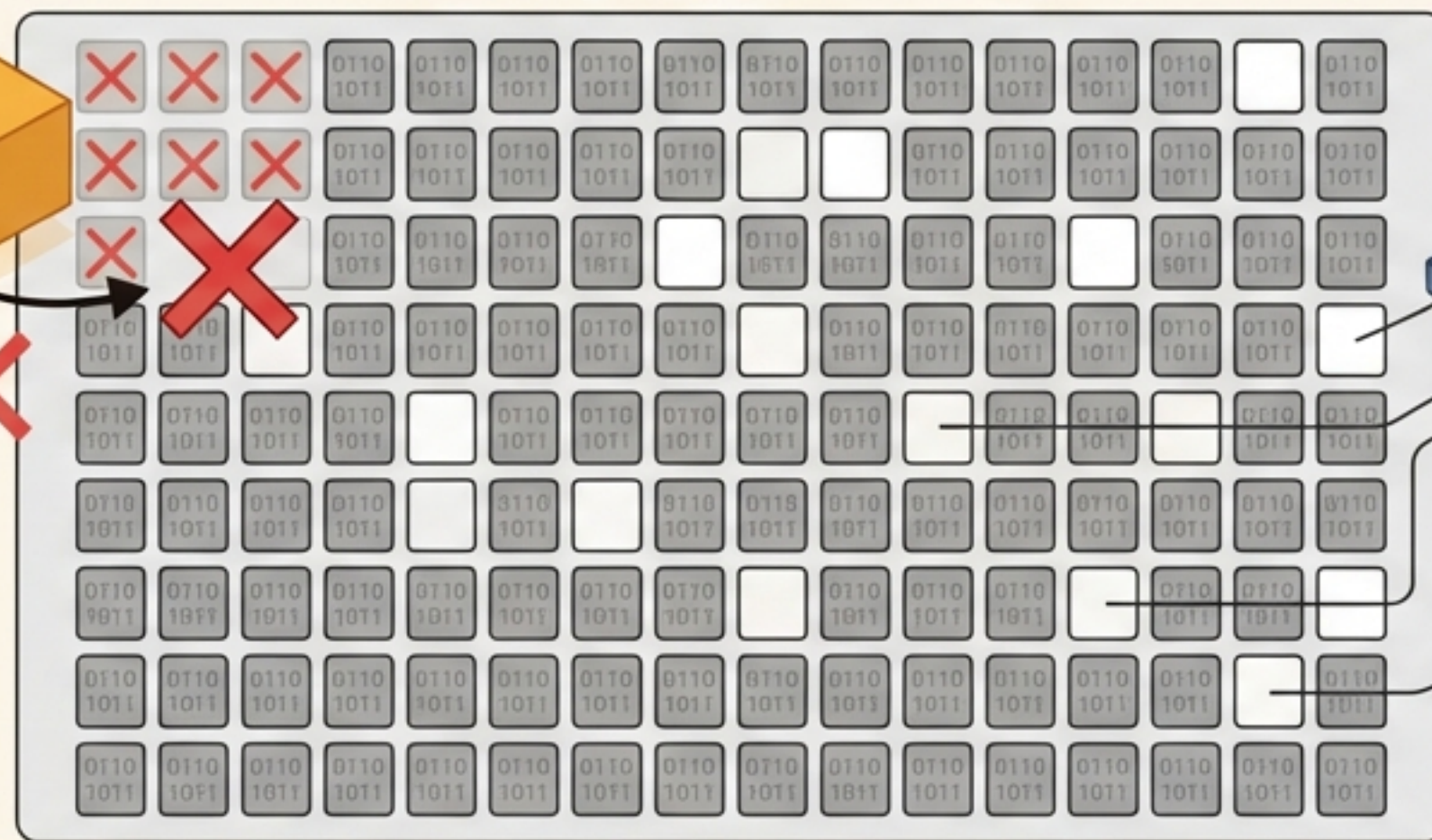
The Limitation of Arrays: The Memory Problem

Array: Contiguous Memory Block
(e.g., [10, 20, 30, 40, 50])



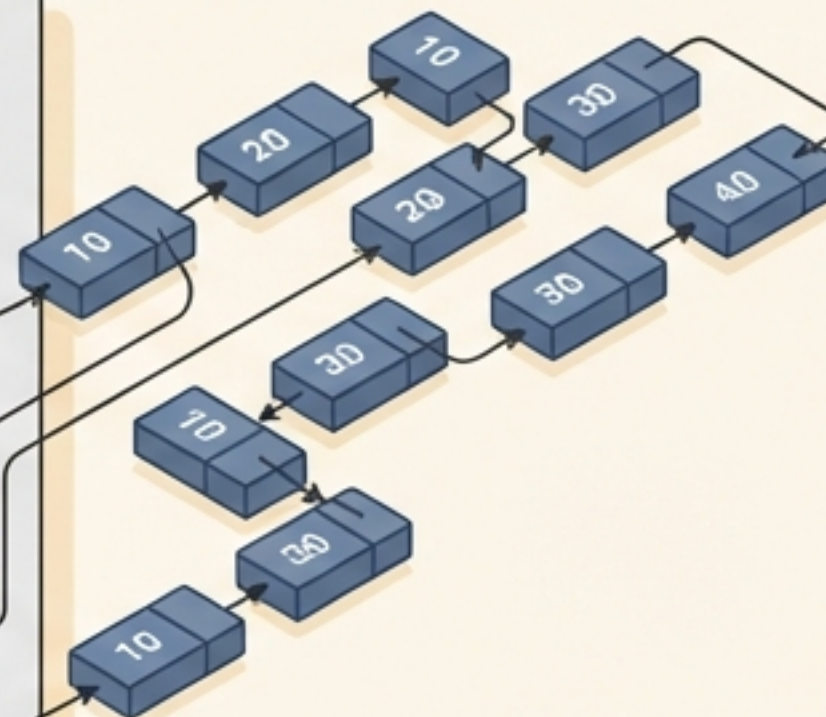
The Array Issue:
Requires a contiguous block of RAM.
If memory is fragmented, allocation fails.

RAM Space (or hard drive)



■ Occupied Memory □ Free Memory

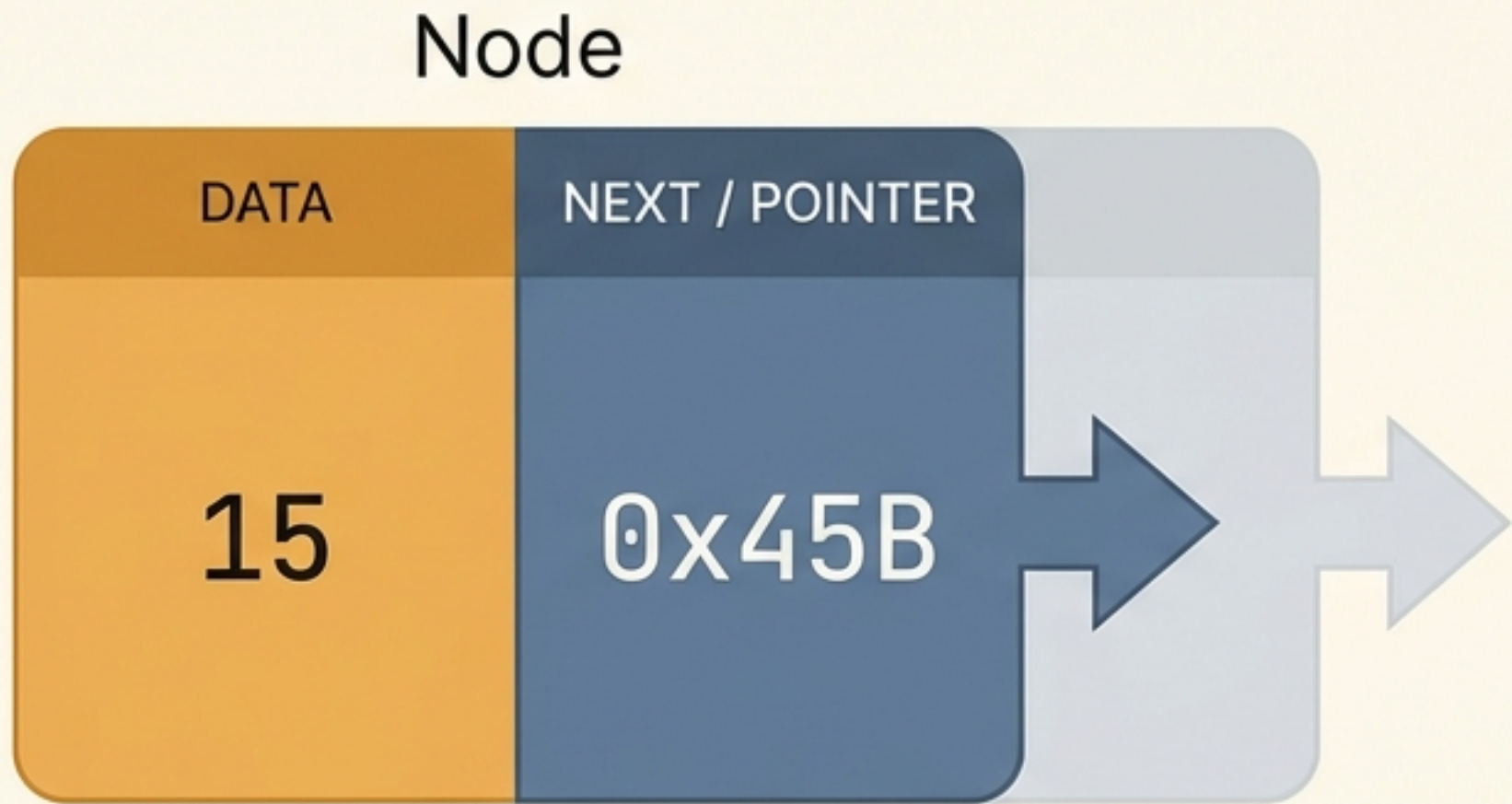
Linked List



The Linked List Solution:
Utilization of scattered memory.

Analogy: An Array is a train (cars physically connected). A Linked List is a treasure hunt (each clue points to the next location).

Anatomy of a Singly Linked List Node



```
class Node:
    def __init__(self, data=None, next=None):
        self.data = data    # The Value
        self.next = next    # The Address
```

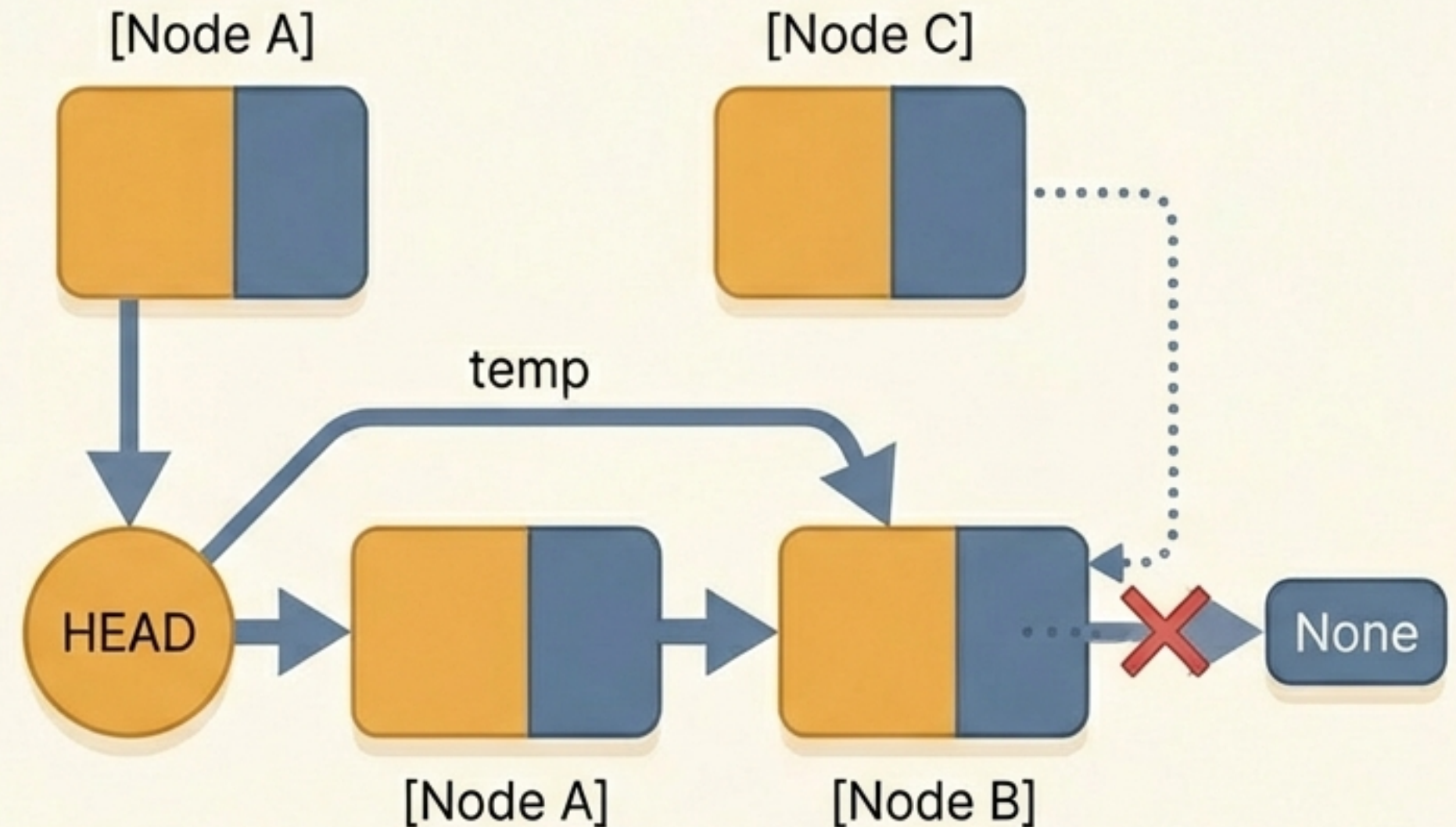
- The Head: The start of the list.
- None/Null: The end of the list.

Building the Chain: The 'Head' and Insertion

Rule #1: The Head is static.
Never move the Head to traverse;
you will lose the list.

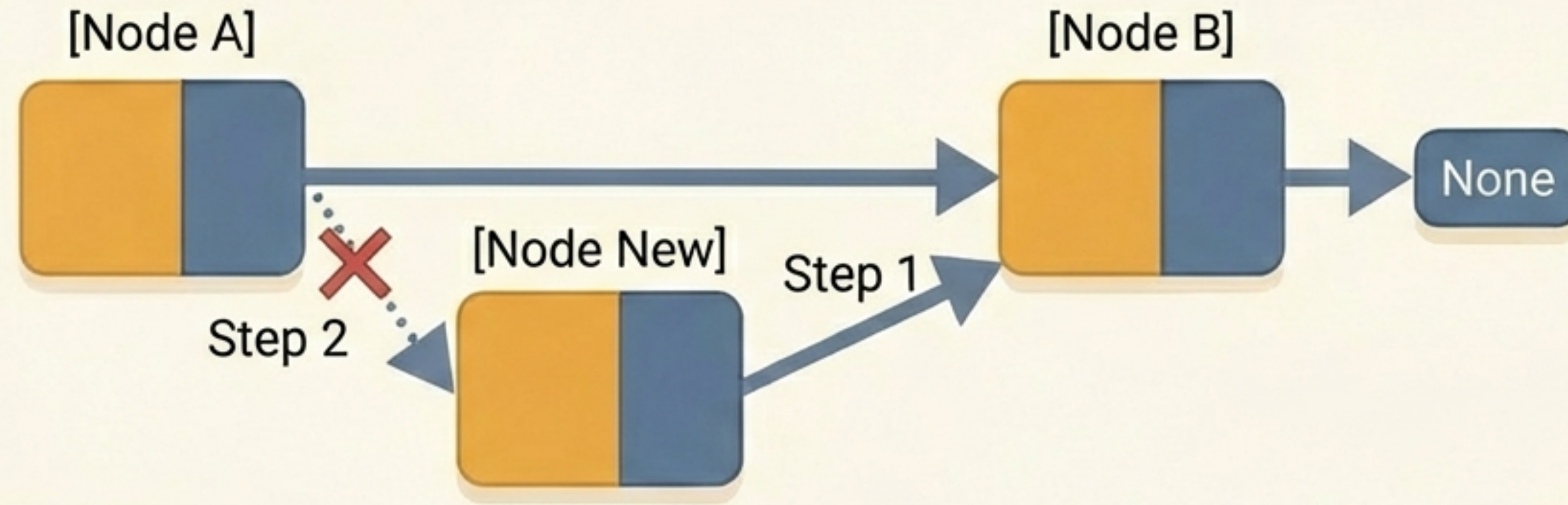
Insertion at End:

1. Create New Node.
2. Use a temporary pointer ("temp") to walk to the end.
3. Update the last node's "next" to point to New Node.



Precision Surgery: Inserting in the Middle

Visual:



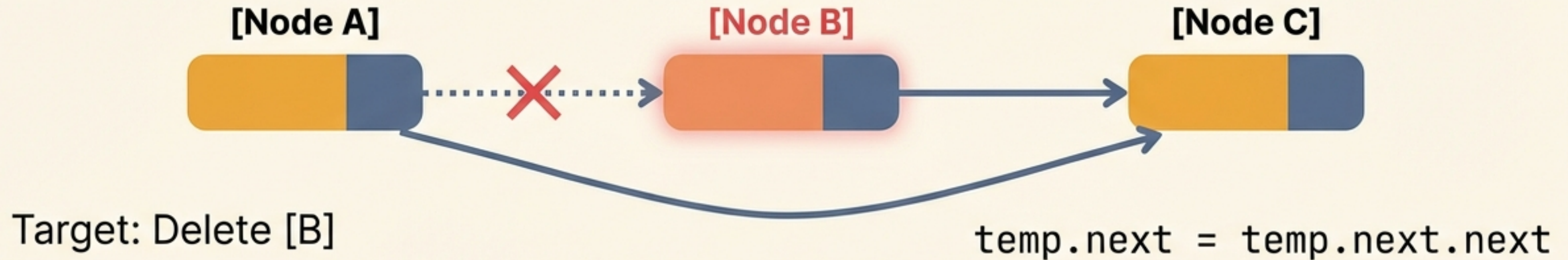
The Goal: Insert 30 after 20.

The Danger: Breaking the chain before reattaching. If you disconnect Node A from Node B before connecting the New Node, the rest of the list is lost.

Correct Order:

1. Connect New Node to the rest of the chain (JetBrains Mono: `New.next = Current.next`).
2. Connect Current Node to New Node (JetBrains Mono: `Current.next = New`).

Deletion & Final Takeaways



Feature	Array	Linked List
Memory	Contiguous (Rigid)	Scattered (Flexible)
Read Access	$O(1)$ Fast	$O(N)$ Slow
Insertion (Mid)	Slow (Shifting needed)	Fast (Pointer change)

Next Step: Practice implementing these classes from scratch on MasterG.co.